# Workout App

Daniel Boyle

Samuel Cardo

Austen Del-Gatto

Taylor Evans

Mobile Application Development

Brayden Girard

## Application Description

This is a workout training app which is a mobile application built with react native and expo and is able to get users to log in with an email and password. Then they can either select a premade workout with set number of exercises, or they can create their own exercises, and they can add in exercise and be able to log each exercise with the number of repetitions, sets and weight. This is a simple but awesome workout app for users to track their workout and to see if they have improved the amount of weight they are able to do or just simply track their workouts.

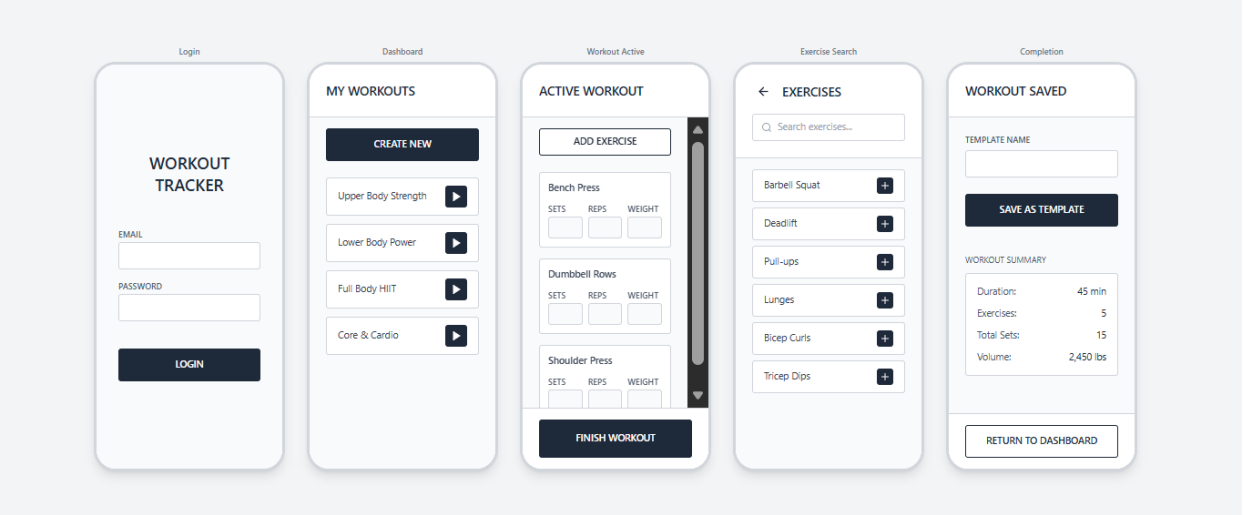
## Target User

The target user of this application is individuals who engage in regular weightlifting activities and want an easy way of tracking their workouts. This includes anyone at any sort of experience level within the gym who wants the ability to logging their exercises, monitor their progress on a specific exercise and be able to use prebuilt workout or create their own workouts.

## Functional and Non-Functional Requirements

| Functional Requirements | Non-Functional Requirements |
|---|---|
| Minimum of four distinct screens | Clear, consistent, and intuitive navigation |
| Stack-based navigation with at least one nested or parameterized navigation flow | Graceful handling of loading, error, and empty states |
| Integration with at least one remote REST API | Responsive layout and usability across screen sizes |
| Persistent data storage using Async Storage or SQLite | Stable behavior with no crashes during normal use |
| User interaction that creates, updates, and displays data | Smooth scrolling of the exercise list and handle 100+ without lag |
| Users can login with an email | Not crash during user interaction |
| Users can create a workout template | Handle empty, loading and error states |
| Users input their repetitions, sets and weight for each exercise | User input should be validated |
| Save completed workouts | |
| navigate to each of the five screens | |

## Wireframes and user flow diagrams



## Component Hierarchy and data flow overview

First Page: Login screen

- User logs in using email and password
- Functionally the user just needs to provide their email, and the backend is checked to link workouts in the database to the email

Second Page: Create workout (Templates?)

- Shows a list of workouts created by the user
- Can each word name? (non-functional)

Third Page: Work out Details

- Choose a workout based on body part/group
- Select target number of reps and weights
- Add to workout
- (Show all workouts added?)

Fourth Page: Workout Selection

- Lists all workouts from Api
- Can search by name or body group

Fifth Page: Workout Complete

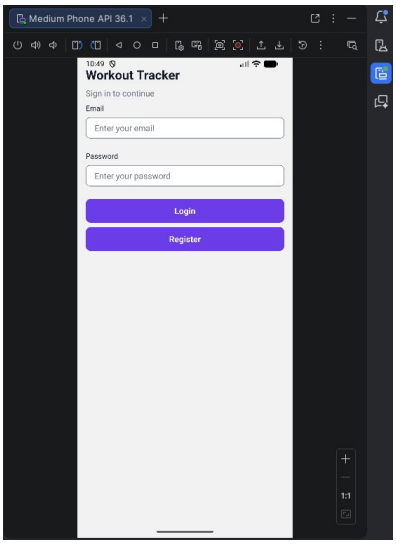
- Option to save workout as a template

- Show Records (Non-Functional)

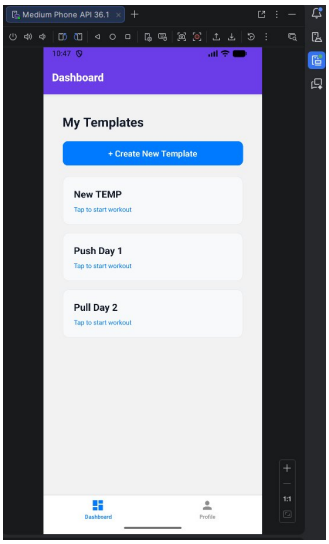
## Test Plan

| Test # | Description | Steps | Expected Result |
|---|---|---|---|
| 1 | Navigate from login to dashboard | Enter email and submit | Dashboard loads |
| 2 | Navigate from Dashboard to work out details | Select workout | Correct layouts for the workout were displayed |
| 3 | Navigate from exercise to details | Select exercise | Data was passed correctly |
| 4 | Save Workout | Complete the workout and enter all the data for each exercise and click save | Workout should be able to view on the workout saved |
| 5 | Load Templates | Open app and go to my workouts page | The workouts should be listed |
| 6 | API Fetch | Go to exercises and view the exercise that were fetched and add one to a template | The API fetch was successful and was able to view the list no problem and added some to a workout template |
| 7 | Enter empty field into email | Click submit | Cause an error to display to tell the user to enter an email in |
| 8 | Entering random garbage into the search bar when selecting an exercise for a template | Navigated to create a workout and typed in 123random | Should say no results were found |
| 9 | App reload | Typing r into the cmd and trying to see if any data or anything like that got messed up on reload | Viewed nothing breaking and the data stayed |

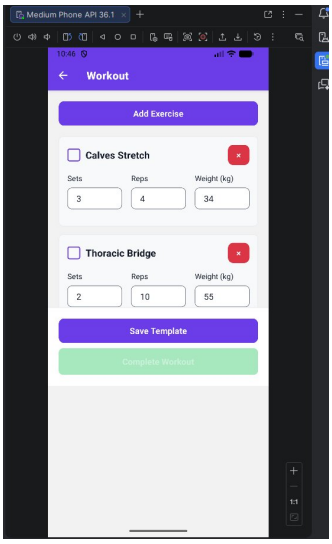
# Screenshots



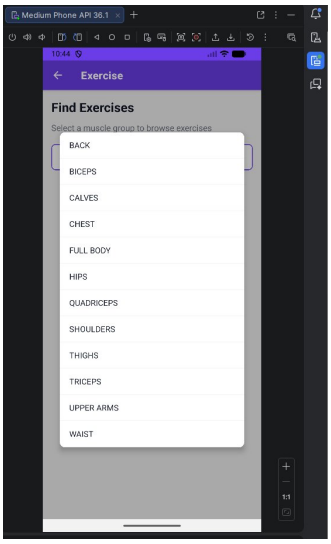

Login Screen



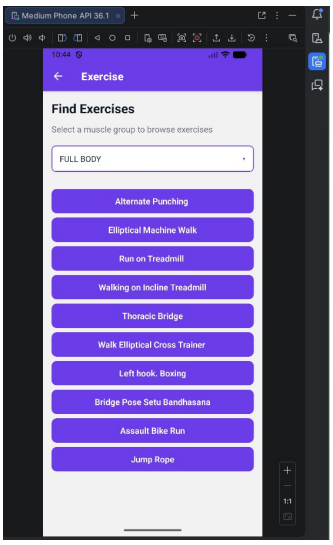

Dashboard

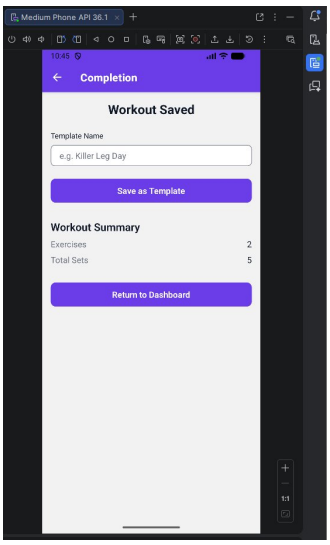



Workout Screen




Exercise Screen - Dropdown



## Description of major design ideas

Some of the major design ideas were about what type of application we wanted to create and all of us threw out some ideas of what we wanted to do for our application and so we decided on making a workout application. Our next decision made was on our authentication approach which we were unsure for a good amount of time but then we made it a good idea of creating a login page and be able to enter in a password and then storing that information in a database to then grab when the user logs in and this way we can keep the data personalized and also kept a simple approach to the authentication. Our next decision was what we wanted for each of our screens and what was going to be displayed on them then the next part we gather idea from other previous designed applications out on the market and gather ideas for the screens like having the login screen and then be able to create your own workout and not just use a prebuilt workout that we have created. Furthermore, we decided what roles each team member was going to play in this group project, so we have Daniel as lead developer, Taylor as our Frontend Developer, Austen as our Data systems engineer and finally me sam as our QA & Documentation Lead. Finally , I final major decision was about our testing strategy was which try and keep the bugs little to none to keep the user from having a poor experience when using our application so each developer went and tested their code each time they created something new and made sure there wasn’t any bugs in it or created new bugs somewhere else.

## Group Member Tasks

### 1. Lead Developer (Architecture & Navigation) - Daniel

This role owns the skeleton of the application. Their immediate priority is ensuring the user can seamlessly move through the five pages.

- **Task 1:** Initialize the Expo React Native project and set up the GitHub repository with strict branch protection rules.
- **Task 2:** Build the React Navigation stack to link all five screens (Login, Dashboard, Workout Details, Exercise Selection, and Workout Complete).
- **Task 3:** Implement the parameterized routing logic to pass the correct data between screens (e.g., passing a selected exercise from Page 4 back to Page 3).
- **Task 4:** Review pull requests from the team to enforce clean code standards and proper modular structure.
- **Task 5:** Write the "description of major design decisions" section for the final design documentation.

## 2. Frontend Developer (UI/UX) - Taylor

This role owns the visual layer. They must ensure the application looks like a cohesive product and handles the non-functional elements gracefully.

- **Task 1.1:** Sketch and finalize the wireframes for the five pages.
- **Task 1.2:** Secure team approval before major coding begins.
- **Task 2:** Build the UI for the Login screen (Page 1) and the reps/weights input fields (Page 3), including clear form validation and error messages.
- **Task 3:** Develop the scrolling list interfaces for the user's saved templates (Page 2) and the API search results (Page 4).
- **Task 4:** Build the static UI elements for the "Name Workout" and "Records" sections on Page 5 so they look polished.
- **Task 5:** Write the "application description and target user" section for the final design documentation.

## 3. Data Systems Engineer (API & Storage) - Austen

This role handles all data movement, ensuring the backend logic satisfies the course's CRUD and external integration requirements.

- **Task 0**: “Wireframe” of database

- **Task 1:** Set up Async Storage or SQLite to handle the local identity management on the Login screen (Page 1).
- **Task 2:** Connect the Exercise Selection screen (Page 4) to a public REST API and write the fetch functions to return exercises based on body group searches.
- **Task 3:** Write the local CRUD functions to save completed workouts (Page 5) and load them onto the Dashboard (Page 2).
- **Task 4:** Bind these data functions to the frontend UI components using the proper React hooks for state and side effects.
- **Task 5:** Map out the "component hierarchy and data flow overview" for the final design documentation.

## 4. QA & Documentation Lead - Sam

This role guarantees the final submission is stable, well-documented, and mathematically meets the 30% grading weight.

- **Task 1:** Draft a comprehensive test plan that specifically targets the data handoffs between the five different pages.
- **Task 2:** Run the application exclusively on the Android Emulator to hunt for bugs, log in any crashes, and ensure the app is stable.
- **Task 3:** Capture the required emulator screenshots of all five pages in a fully functional state.
- **Task 4:** Compile the design documents, test plan, and bug resolution summary into a single, cohesive package.

- **Task 5:** Write the final contribution summary and verify the project runs perfectly from a clean install before creating the final ZIP file.

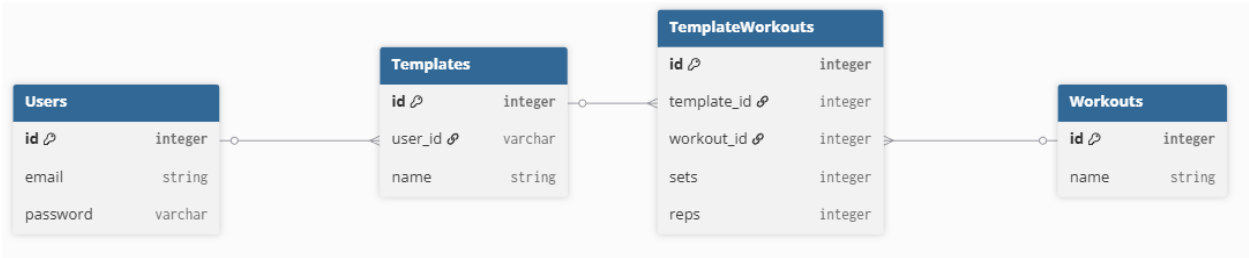
Database Design:

```
Table: Users {

  id integer [primary key]

  email string

  password varchar

}

Table: Templates {

  id integer [primary key]

  user_id varchar

  name string

}



Table: Workouts {

  id integer [primary key]
```

```
  name string

}



Table: TemplateWorkouts {

  id integer [primary key]

  template_id integer

  workout_id integer

  sets integer

  reps integer

}

Ref user_template: Templates.user_id > Users.id // many-to-one

Ref: Workouts.id < TemplateWorkouts.workout_id

Ref: Templates.id < TemplateWorkouts.template_id

//swap? Whichever looks better
```

- User Table
    - ID int Primary Key
    - Email String

    - Password String
- Templates
    - ID int Primary Key
    - User_ID int Foreign Key
    - Name String
- Workouts
    - ID int Primary Key
    - Name String
- TemplateWorkouts
    - Id int Primary Key
    - Template_ID int Foreign Key
    - Workout_ID int Foreign Key
    - Sets
    - Reps



# Bugs

Bug#156 Limitation - Medium

Android Studio – Pixel 9

To reproduce the bugs just enter a very large email address and see that there is no validation on it.

This bug is for the input fields I found this when testing the application and the user is able to type in a very large set of text into for example the email and password I was able to input a very large email which was over 100 characters, and this is a unrealistic email length.

Bug #157: Workout screen not refreshing after returning from exercise selection

Added `useFocusEffect` to workout.tsx so the exercise list reloads from the database every time the screen comes into focus, not just on initial mount.

Bug #158: Exercise add failing with database group/NOT NULL errors

The code was trying to read `exercise.bodyPart` from the API response, but that field doesn't exist. Fixed by passing the user's search term (`bodyPart` state) into `createOrGetExercise` instead.

Bug #159: Workout completion not saving updated sets/reps/weight

The "Complete Workout" button navigated directly to the completion screen without persisting data. Extracted the save logic into a shared `saveAllExercises()` helper and called it from `handleCompleteWorkout` before navigating.

Bug #160: Completion screen showing unnecessary save prompt

The modal triggered automatically on arrival, which was redundant after the workout was already saved. Removed the modal and its state entirely, leaving just the congratulations message and a "Back to Dashboard" button.

Bug #161: Exercise search used a free-text field with no guidance on valid values

The API only accepts 12 specific `bodyParts` values (e.g. `CHEST`, `FULL BODY`), which a user typing freely would not know. Replaced the input with a chip grid, then later a dropdown, so only valid values can be selected — also eliminating the need for a separate "Search" button by firing the search on selection.

Bug #162: API field name mismatch (`bodyPart` vs `bodyParts`)

Confirmed via live API inspection that each exercise object returns `bodyParts` (an array), not the singular `bodyPart`. The query parameter for filtering is also `bodyParts`, not `bodyPart`. Code was updated to reflect the correct field name.

Bug #163: Exercise results screen returned a React key warning and user could not see results properly

The keys were switched to reflect results from the API and flow mapping was fixed. Flex was added to the screen container style to properly show results.

Bug #164: Results only show 10 exercises per body part

The API call limits to only 10 exercises per search. Currently not fixed for app simplicity, but otherwise can be fixed by increasing the limit of the API query.

Bug #165: API field name mismatch (`bodyPart` vs `bodyParts`)

Confirmed via live API inspection that each exercise object returns `bodyParts` (an array), not the singular `bodyPart`. The query parameter for filtering is also `bodyParts=`, not `bodyPart=`. Code was updated to reflect the correct field name.

Bug #165: Exercise results screen returned a React key warning and user could not see results properly

The keys were switched to reflect results from the API and flow mapping was fixed. Flex was added to the screen container style to properly show results.